

## CO2-AT2- Scenario Based Questions

**Name :** S.Deepadharshan

**Reg No :** 192472046

**Course Code :** ITA0513

**Course Name :** Computer Vision

**Questions :**

### **Q3: Spatial Filtering (Image Smoothing)**

- Scenario: Medical technicians notice that brain MRI scans are heavily contaminated with sharp, isolated "salt-and-pepper" noise artifacts caused by sensor interference. You must remove this noise completely without blurring the critical edges of internal brain structures.

### **Q8: Image Segmentation (Thresholding)**

- Scenario: You are developing an OCR pipeline to digitize archival historical text documents. The paper background is unevenly lit with heavy dark shadows across the page corners. A single global threshold fails, so you need to dynamically compute thresholds for local neighborhoods.

### **Q3. SPATIAL FILTERING (IMAGE SMOOTHING)**

#### **Objective :**

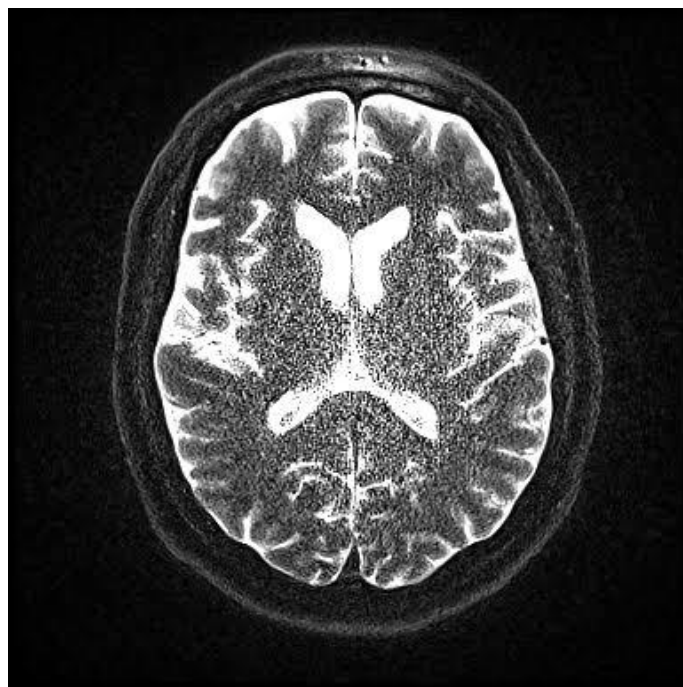
To remove Salt-and-Pepper noise from a brain MRI image using a Median Filter while preserving the important edges and structural details of the brain.

#### **Tool Used :**

Python, OpenCV, NumPy and Visual Studio Code.

#### **Input Image :**

A Brain MRI image is used as the input image. Salt-and-Pepper noise is artificially introduced into the MRI image to simulate sensor interference. A  $5 \times 5$  Median Filter is then applied to remove the noise.



**Figure 1: Original Brain MRI Image**

#### **Procedure :**

1. Read the Brain MRI input image using OpenCV.
2. Create a copy of the original image for adding noise.
3. Generate random pixel locations using NumPy.
4. Introduce Salt noise by changing selected pixels to white.
5. Introduce Pepper noise by changing selected pixels to black.
6. Add approximately 10% Salt-and-Pepper noise to the MRI image.
7. Apply a  $5 \times 5$  Median Filter using `cv2.medianBlur()`.
8. Display the original, noisy and filtered MRI images.

9. Save the noisy and filtered images for comparison.

**Program :**

```
import cv2
import numpy as np
import os
# Create output folder
os.makedirs("outputs", exist_ok=True)
# Read MRI image
image = cv2.imread("images/mri.jpg")
if image is None:
    print("Error: mri.jpg not found!")
    exit()
# Create a copy for adding noise
noisy = image.copy()
# Add Salt and Pepper noise
noise_percentage = 0.10
random_values = np.random.rand(
    image.shape[0],
    image.shape[1]
)
noisy[random_values < noise_percentage / 2] = 0
# Salt noise - white pixels
noisy[random_values > 1 - noise_percentage / 2] = 255
# Apply 5x5 Median Filter
filtered = cv2.medianBlur(noisy, 5)
# Save outputs
cv2.imwrite("outputs/q3_noisy.jpg", noisy)
cv2.imwrite("outputs/q3_filtered.jpg", filtered)
# Display images
cv2.imshow("Original MRI", image)
```

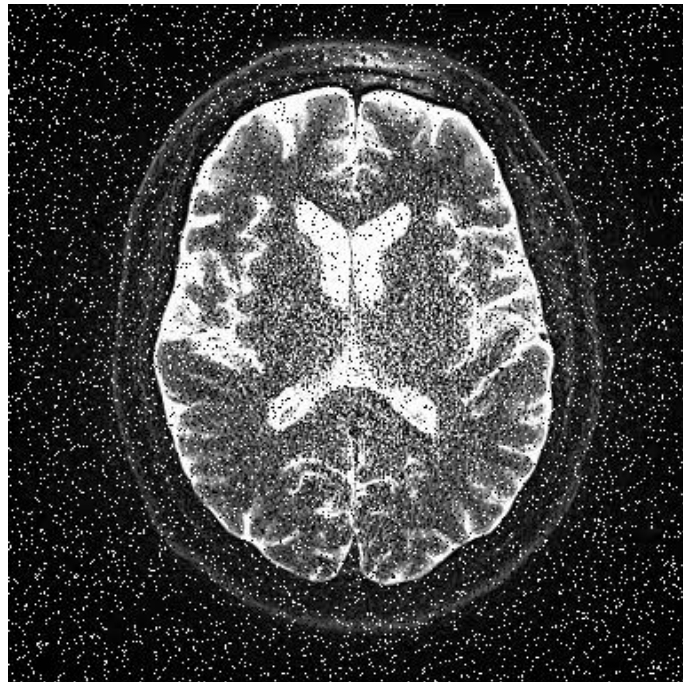
```
cv2.imshow("MRI with Salt and Pepper Noise", noisy)
```

```
cv2.imshow("Median Filtered MRI", filtered)
```

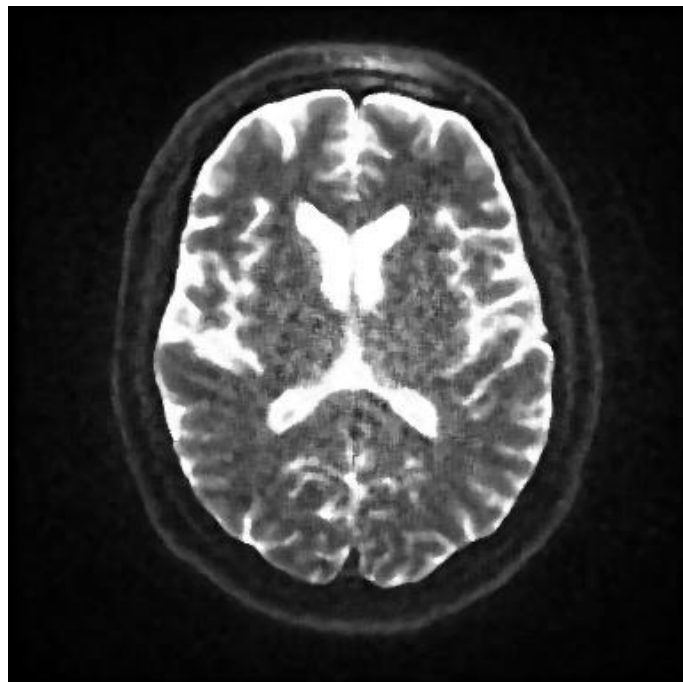
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

**Output :**



**Figure 2: Brain MRI Image with Salt-and-Pepper Noise**



**Figure 3: MRI Image after  $5 \times 5$  Median Filtering**

**Analysis :**

Salt-and-Pepper noise appears as randomly distributed black and white pixels in the MRI image. This type of noise can reduce image quality and make important internal structures more difficult to observe.

The Median Filter is a non-linear spatial filtering technique. It examines the pixels within a local neighbourhood, arranges their intensity values and replaces the centre pixel with the median value.

In this experiment, a  $5 \times 5$  neighbourhood is used. The isolated black and white noisy pixels are effectively removed because extreme intensity values have less influence on the median.

Unlike averaging filters, the Median Filter removes impulse noise without significantly blurring important edges. This is particularly useful for medical images because the boundaries of internal brain structures should be preserved.

The output shows that most of the Salt-and-Pepper noise is removed while the important structural information of the MRI image remains visible.

**Result :**

Salt-and-Pepper noise was successfully introduced into the Brain MRI image and removed using a  $5 \times 5$  Median Filter. The Median Filter effectively reduced the noise while preserving the important edges and structural details of the MRI image.

## Q8. IMAGE SEGMENTATION (THRESHOLDING)

### Objective :

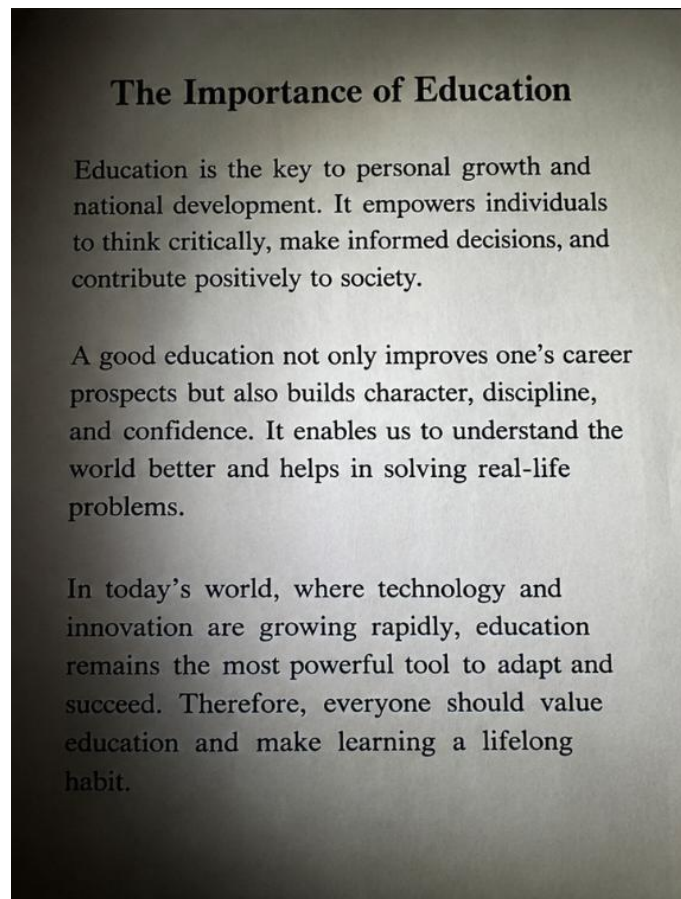
To segment text from a historical document affected by uneven illumination and shadows using Adaptive Gaussian Thresholding, so that the foreground text can be clearly separated from the background.

### Tool Used :

Python, OpenCV and Visual Studio Code.

### Input Image :

A document image containing text with uneven lighting and dark shadow regions is used as the input image. Adaptive thresholding is applied to calculate different threshold values for different local regions of the document.



**Figure 1: Original Document Image with Uneven Illumination**

### Procedure :

1. Read the document image in grayscale using OpenCV.
2. Apply a slight  $3 \times 3$  Gaussian Blur to reduce small image noise.
3. Select an odd local neighbourhood block size of 11.
4. Apply Adaptive Gaussian Thresholding using `cv2.adaptiveThreshold()`.

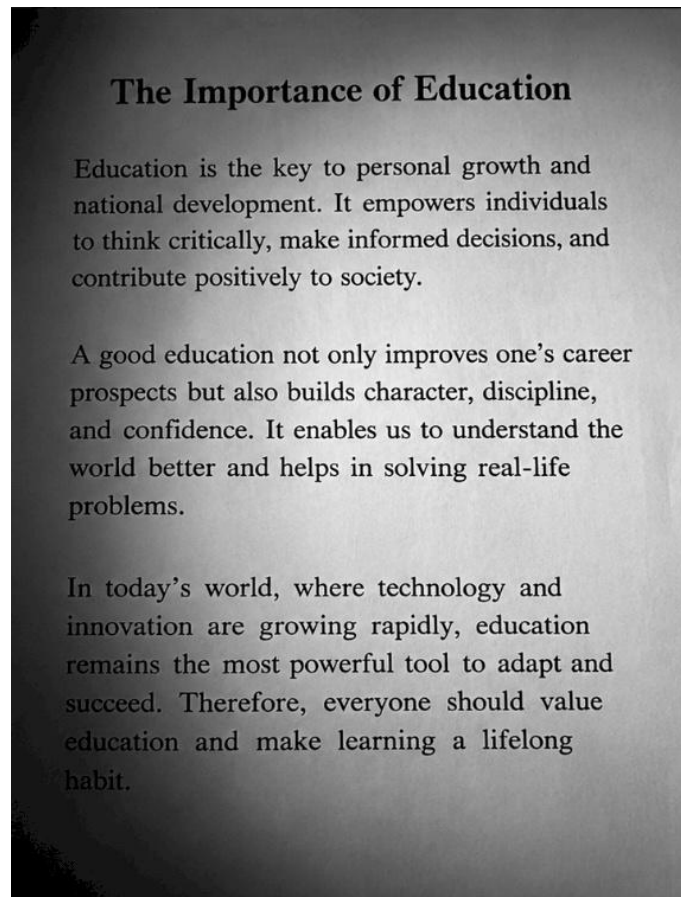
5. Set the maximum binary intensity value to 255.
6. Use a constant value  $C = 2$  to adjust the locally calculated threshold.
7. Convert the document into a binary image containing a white background and dark text.
8. Display the original grayscale document and the segmented output.
9. Save the processed images for comparison.

**Program :**

```
import cv2
import os
# Create outputs folder
os.makedirs("outputs", exist_ok=True)
# Read document image in grayscale
image = cv2.imread(
    "images/document.jpg",
    cv2.IMREAD_GRAYSCALE
)
if image is None:
    print("Error: document.jpg not found!")
    exit()
# Apply slight Gaussian blur
blurred = cv2.GaussianBlur(image, (3, 3), 0)
# Apply Adaptive Gaussian Thresholding
binary = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)
# Save outputs
```

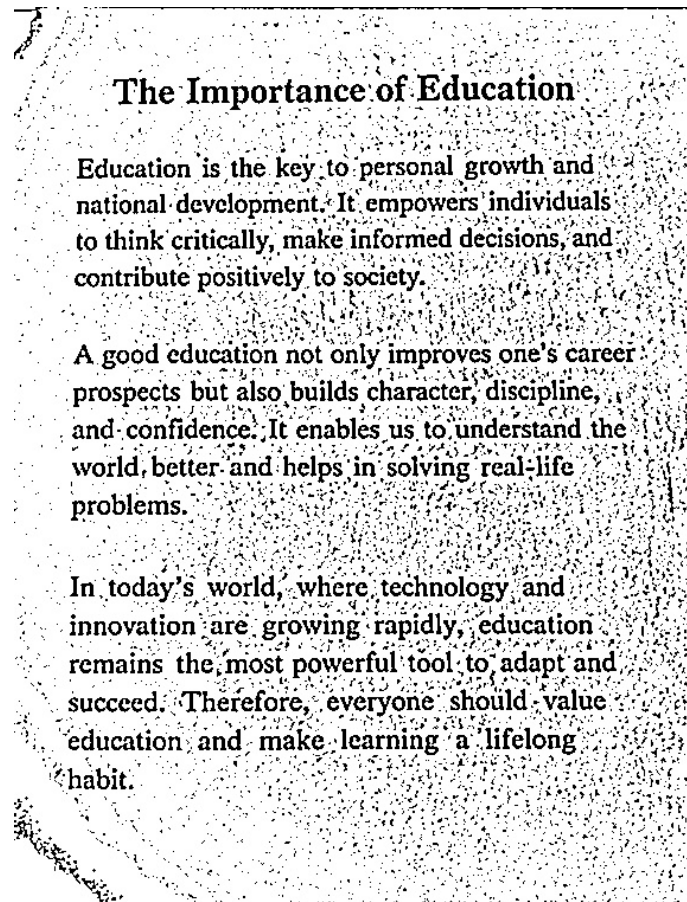
```
cv2.imwrite("outputs/q8_grayscale.jpg", image)
cv2.imwrite("outputs/q8_threshold.jpg", binary)
# Display results
cv2.imshow("Original Document", image)
cv2.imshow("Adaptive Threshold Output", binary)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**Output :**



**Figure 2: Original Document in Grayscale**





**Figure 3: Adaptive Thresholding Output**

#### **Analysis :**

A single global threshold value may not correctly segment a document when different parts of the page have different brightness levels. Text present in darker regions may disappear, while shadows may incorrectly appear as foreground regions.

In this experiment, Adaptive Gaussian Thresholding uses an  $11 \times 11$  local neighbourhood. The neighbouring pixels are Gaussian-weighted to determine the local threshold. The constant value  $C = 2$  is subtracted from the calculated threshold to improve the separation between the text and background.

The resulting binary image shows the text as dark foreground regions against a mostly white background. The effect of uneven illumination and shadows is significantly reduced, making the document more suitable for further processing such as Optical Character Recognition (OCR).

#### **Result :**

The unevenly illuminated document image was successfully segmented using Adaptive Gaussian Thresholding. The method effectively reduced the influence of local shadows and separated the foreground text from the background, producing a clear binary image suitable for OCR applications.